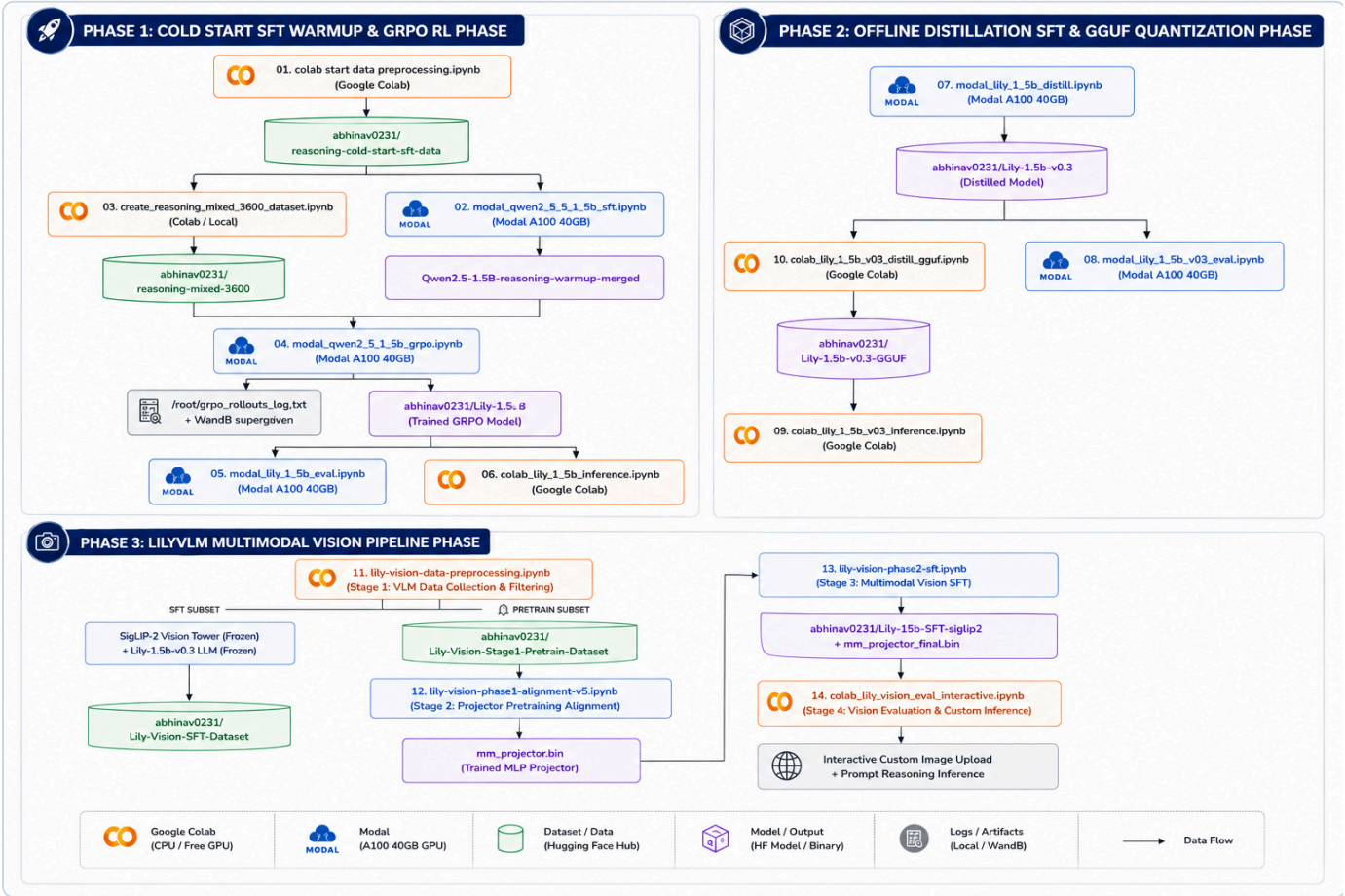


AI Training Handbook

Executive Summary & Engineering Architecture

This handbook serves as the definitive technical reference manual for our LLM and Vision-Language Model (VLM) training ecosystem. Synthesized directly from the production code across all **16 notebooks** in our codebase, it bridges theoretical deep-learning concepts with real-world implementation technicalities.



The 16-Notebook End-to-End Development Lifecycle

```
[ Phase 1: Text LLM Foundations & SFT Warmup ]
├─ 01_cold_start_data_preprocessing.ipynb      (Data Curation: ChatML Format, Reasoning Datasets)
├─ 02_modal_qwen_2_5_1_5b_sft.ipynb           (Warmup SFT: Qwen2.5-1.5B on Modal A100 -> Warmup Merged)

[ Phase 2: Reinforcement Learning via GRPO ]
├─ 03_create_reasoning_mixed_3600_dataset.ipynb (Curate 3.6k Multi-Domain Reasoning Dataset)
├─ 04_modal_qwen_2_5_1_5b_grpo.ipynb           (GRPO RL Training: 4 Domain Rewards, DAPO Clipping -> Lily-1.5B v0.1)
├─ 05_modal_lily_1_5b_eval.ipynb               (Benchmark Evaluation: lm-eval Harness on Modal A100/L4)
├─ 06_colab_lily_1_5b_inference.ipynb           (Fast Colab GPU Inference: Unsloth Kernels & Dual-Channel Parsing)

[ Phase 3: Offline CoT Distillation & Text Quantization ]
├─ 07_modal_lily_1_5b_distill.ipynb             (Offline Distillation: Sarvam-105B -> Lily-1.5b-v0.3 w/ Sequence Packing)
├─ 08_modal_lily_1_5b_v03_eval.ipynb           (Benchmark Evaluation: lm-eval Harness for Lily-1.5b-v0.3)
├─ 09_colab_lily_1_5b_v03_inference.ipynb       (Native Transformers Inference & Schema Compliance Auditor)
├─ 10_colab_lily_1_5b_v03_distill_gguf.ipynb    (llama.cpp Compilation, Jinja Template Patching, K-Quantization)

[ Phase 4: Multimodal Projector Alignment & Pretraining ]
├─ 11_colab_lily-vision-pretraining-data-preprocessing.ipynb (Multi-Dataset Alignment Data: LAION, AI2D, ChartQA, DocVQA)
├─ 12_modal_lily-vision-projector-pretraining.ipynb (Phase 1 Alignment: SigLIP-2 + MLP Projector Training -> v0.4 Projector)

[ Phase 5: Multimodal SFT, Interactive Eval & Edge Deployment ]
├─ 13_colab_lily_vision_sft_data_preprocessing.ipynb (MMR1-SFT + Filtered Sarvam-105B Data Packaging: 64k Samples)
├─ 14_modal_lily_vision_sft_training.ipynb         (Phase 2 Multimodal SFT: LoRA + Projector Training -> Lily-1.5b-v0.5)
├─ 15_colab_lily_vision_eval_interactive.ipynb     (Multimodal Evaluation & Gradio/Interactive Colab REPL)
├─ 16_colab_lily_vlm_gguf.ipynb                   (Multimodal llama.cpp Export: LLM Backbone GGUF + SigLIP2 mmproj)
```

1. Fine-Tuning Foundations: PEFT & LoRA Technical Deep-Dive

1.1 Mathematical Formulation of Low-Rank Adaptation (LoRA)

In standard full fine-tuning, an LLM parameter matrix $W_0 \in \mathbb{R}^{d \times k}$ is updated directly via gradient descent ($W = W_0 + \Delta W$). When d and k are large (e.g., hidden dimension $d = 1536$ and intermediate MLP dimension $k = 8960$ in Qwen2.5-1.5B), updating ΔW directly requires massive VRAM for storing optimizer states (AdamW requires 8 bytes per parameter for first and second moments).

Explicit Variable Definitions:

- $W_0 \in \mathbb{R}^{d \times k}$: Pre-trained frozen base weight matrix of the target linear layer.
- d : Input feature dimension (in-features / hidden size, e.g., $d = 1536$ in Qwen2.5-1.5B).
- k : Output feature dimension (out-features / projection size, e.g., $k = 8960$ in SwiGLU MLP layers).
- $\Delta W \in \mathbb{R}^{d \times k}$: High-dimensional weight update matrix resulting from fine-tuning gradients ($W = W_0 + \Delta W$).

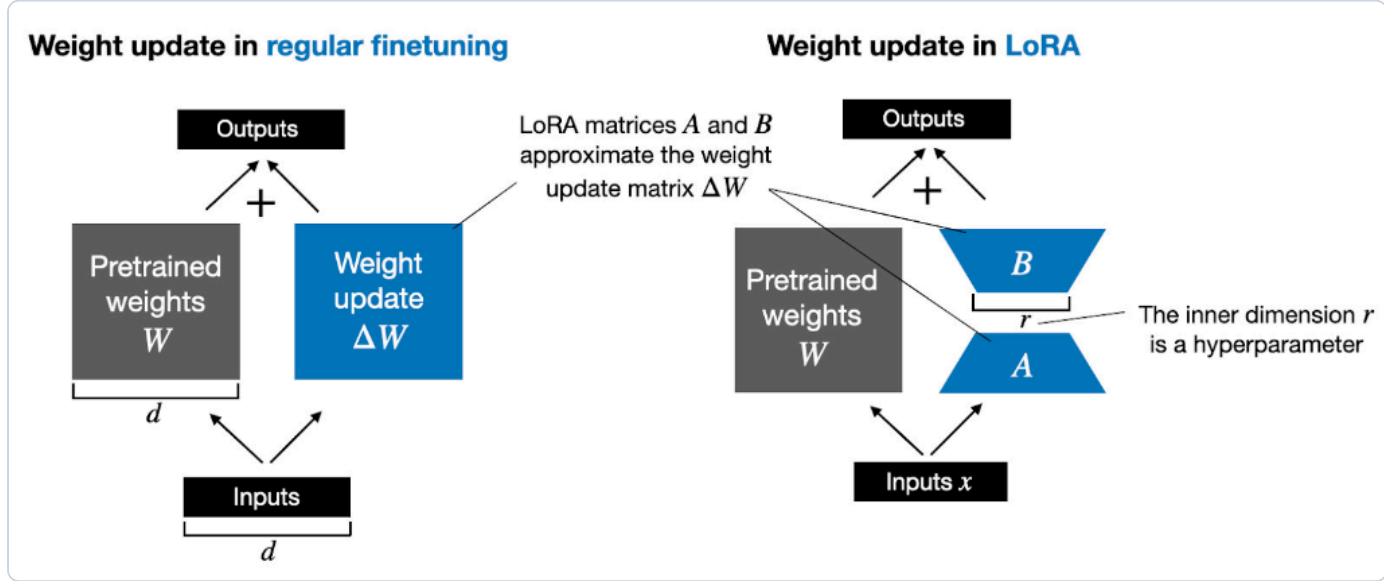
- r : Inner low-rank hyperparameter ($r \ll \min(d, k)$).
- $A \in \mathbb{R}^{r \times k}$: Down-projection adapter matrix, initialized with Gaussian distribution $\mathcal{N}(0, \frac{1}{r})$.
- $B \in \mathbb{R}^{d \times r}$: Up-projection adapter matrix, initialized to zero ($B = 0$) ensuring $\Delta W = 0$ at step 0.
- α : Constant LoRA scaling hyperparameter. The ratio $\frac{\alpha}{r}$ scales the adapter's contribution relative to the base model.

LoRA parametrizes the weight update matrix ΔW by decomposing it into two low-rank matrices:

$$\Delta W = B \cdot A$$

During forward propagation, the linear layer output h for an input x is computed as:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} (B \cdot A x)$$



1.2 QLoRA: NormalFloat4 (NF4) Base Weight Quantization

In QLoRA, the base model weights W_0 are quantized to 4-bit NormalFloat (NF4) precision, while the adapter matrices A and B remain in 16-bit precision (BF16 or FP16). NF4 is an information-theoretically optimal quantile quantization scheme for normally distributed weights:

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k} \right) + Q_X \left(\frac{i+1}{2^k} \right) \right)$$

where $Q_X(\cdot)$ is the quantile function of the standard normal distribution $\mathcal{N}(0, 1)$.

1.3 Target Module Definitions & Code Implementation

Across our codebase (`02_modal_qwen_2_5_1_5b_sft.ipynb` , `04_modal_qwen_2_5_1_5b_grpo.ipynb` , `07_modal_lily_1_5b_distill.ipynb` , `14_modal_lily_vision_sft_training.ipynb`), we apply LoRA across all 7 linear projection layers of the Transformer architecture:

```

from unsloth import FastLanguageModel

model = FastLanguageModel.get_peft_model(
    model,
    r = 32,                      # Rank: 32 (SFT/Distill/Vision), 64 (GRPO RL)
    lora_alpha = 64,             # Alpha: 32 (SFT), 64 (GRPO/Distill/Vision)
    target_modules = [
        "q_proj", "k_proj", "v_proj", "o_proj",      # Self-Attention Projections
        "gate_proj", "up_proj", "down_proj"          # SwiGLU MLP Projections
    ],
    lora_dropout = 0,            # 0 is optimized for Unsloth kernel fusion
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)

```

Detailed Technical Explanation of Target Modules:

- **q_proj (Query Projection)**: Transforms input hidden states into Query matrices (Q) for multi-head self-attention.
- **k_proj (Key Projection)**: Transforms input hidden states into Key matrices (K) for computing attention dot-product scores (QK^T).
- **v_proj (Value Projection)**: Transforms input hidden states into Value matrices (V) carrying contextual token information.
- **o_proj (Output Projection)**: Projects concatenated multi-head attention outputs back to the model hidden dimension ($d = 1536$).
- **gate_proj (Gate Projection in SwiGLU)**: Computes the element-wise gating signal in SwiGLU feed-forward networks ($\text{Swish}(xW_{\text{gate}}) \otimes xW_{\text{up}}$).
- **up_proj (Up Projection in SwiGLU)**: Expands input hidden dimension to high-dimensional intermediate space ($1536 \rightarrow 8960$).
- **down_proj (Down Projection in SwiGLU)**: Contracts intermediate dimension back to model hidden dimension ($8960 \rightarrow 1536$).

Trainable Parameter Efficiency Calculation

For Qwen2.5-1.5B ($d_{\text{model}} = 1536$, $N_{\text{layers}} = 28$): - Total Base Parameters: 1,580,643,840 (~1.58B) - Trainable Adapter Parameters ($r = 32$): 36,929,536 (~36.9M) - **Trainable Ratio**: $\frac{36.9\text{M}}{1580.6\text{M}} \approx 2.34\%$ to 4.0%

1.4 Lossless 16-Bit Adapter Merging

Before running RL (GRPO), performing offline distillation, or exporting to GGUF, adapter weights must be mathematically merged into the base model to eliminate dual-path inference overhead:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r}(B \cdot A)$$

In code (02 , 04 , 07 , 14):

```

model_m, tok_m = FastLanguageModel.from_pretrained(
    model_name = OUTPUT_DIR,
    max_seq_length = MAX_SEQ_LENGTH,
    dtype = torch.bfloat16, # Full precision BF16 merge
    load_in_4bit = False,    # Prevents quantization artifacts during merge
)

model_m.save_pretrained_merged(MERGED_DIR, tok_m, save_method="merged_16bit")
model_m.push_to_hub_merged(HF_MERGED_REPO, tok_m, save_method="merged_16bit", token=HF_TOKEN)

```

2. Hyperparameter Engineering & Optimization Strategy

2.1 The Global Effective Batch Size & Step Calculation

1. What is a Training Step?

A **single optimizer step** corresponds to one iteration where model weights are updated by the optimizer using calculated gradients.

2. Micro-Batches vs. Gradient Accumulation

To fit large models into GPU memory, training datasets are split into smaller **micro-batches** (B_{device}). When gradient accumulation is enabled (N_{accum}), the engine performs N_{accum} forward and backward passes, accumulating gradients in VRAM, before executing a single `optimizer.step()` update.

3. Mathematical Formula for Effective Batch Size:

$$B_{\text{eff}} = B_{\text{device}} \times N_{\text{accum}} \times N_{\text{GPU}}$$

where: - B_{device} : Per-device micro-batch size processed in a single forward pass per GPU. - N_{accum} : Number of micro-batches accumulated before updating model weights. - N_{GPU} : Number of parallel GPUs participating in training.

4. Mathematical Formula for Total Optimizer Steps:

$$\text{Total Optimizer Steps} = \frac{|\mathcal{D}| \times N_{\text{epochs}}}{B_{\text{eff}}}$$

where $|\mathcal{D}|$ is the number of samples in the dataset and N_{epochs} is the total number of training passes over the dataset.

Concrete Worked Step Calculation Examples from Codebase:

- **SFT Warmup (Notebook 02)**: 4,000 samples $\times$ 2 Epochs = 8,000 total samples. $B_{\text{eff}} = 8 \times 2 \times 1 = 16$.
Total Steps = $\frac{8000}{16} = 500$ steps.
- **Distillation (Notebook 07)**: 91,457 raw samples packed into 33,297 contiguous sequences $\times$ 2 Epochs = 66,594 sequence passes. $B_{\text{eff}} = 24 \times 1 \times 1 = 24$. Total Steps = $\frac{66594}{24} = 2,776$ steps.
- **Vision Phase 1 Alignment (Notebook 12)**: 121,237 samples $\times$ 1 Epoch. $B_{\text{eff}} = 16 \times 3 \times 1 = 48$.
Total Steps = $\frac{121237}{48} \approx 2,525$ steps.
- **Vision Phase 2 SFT (Notebook 14)**: 64,000 samples $\times$ 1 Epoch. $B_{\text{eff}} = 24 \times 2 \times 1 = 48$. Total Steps = $\frac{64000}{48} \approx 1,333$ steps.

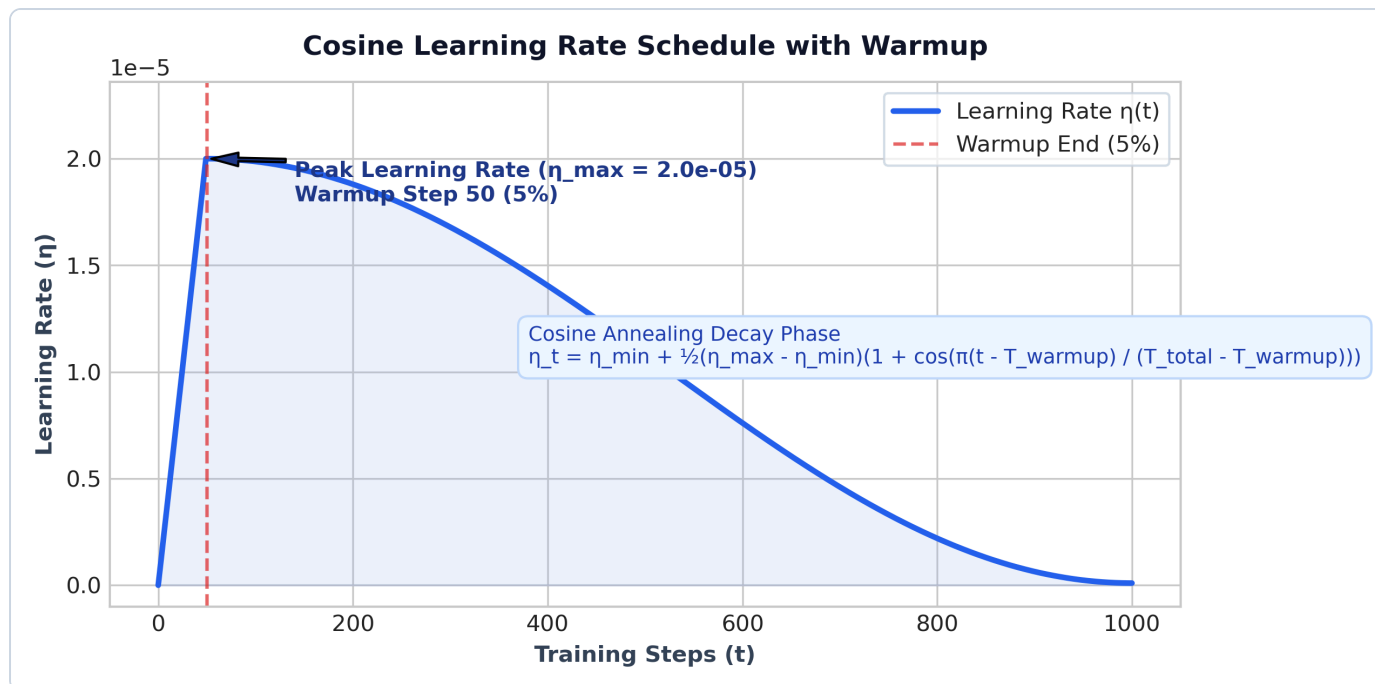
2.2 Verified Production Hyperparameter Comparison Matrix

Hyperparameter	SFT Warmup (02)	GRPO RL (04)	Distillation (07)	Vision Phase 1 (12)	Vision Phase 2 (14)
Base Model	Qwen2.5-1.5B-Instruct	Qwen2.5-Warmup-Merged	Lily-1.5B (v0.1)	Lily-1.5b-v0.3	Lily-1.5b-v0.3
Dataset Size ($\mathcal{D}$)	4,000 samples	3,600 prompts	91,457 (train)	121,237 samples	64,000 samples
Epochs / Steps	2 Epochs (500 steps)	600 Max Steps	2 Epochs (2,776 steps)	1 Epoch (~2,525 steps)	1 Epoch (~1,333 steps)
Per-Device Batch (B_{device})	8	4	24	16	24
Grad Accum (N_{accum})	2	4	1	3	2
Effective Batch (B_{eff})	16	16	24	48	48
Peak Learning Rate (η_{max})	1×10^{-5}	5×10^{-6}	2×10^{-5}	2×10^{-4}	2×10^{-5}
LR Scheduler	Cosine	Cosine	Cosine	Cosine w/ warmup	Cosine w/ warmup
Warmup Ratio	0.05	0.05	0.03	0.05	0.03
Optimizer	adamw_torch_fused	adamw_torch_fused	adamw_torch_fused	AdamW ($\beta_1 = .9, \beta_2 = .95$)	AdamW
Weight Decay	0.01	0.0	0.0	0.05	0.01
Max Sequence Length	3,072	3,072	4,096	256	2,048
Sequence Packing	False	False	True	False	False (LengthGrouped)
Hardware	Modal A100 40GB	Modal A100 40GB	Modal A100 40GB	Modal A100 40GB	Modal A100 40GB

2.3 Learning Rate Schedules & Mathematical Formulation

We utilize Cosine Annealing with Warmup across all training runs.

$$\eta_t = \begin{cases} \eta_{\text{max}} \cdot \frac{t}{T_{\text{warmup}}}, & t \leq T_{\text{warmup}} \\ \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos \left(\frac{\pi(t - T_{\text{warmup}})}{T_{\text{total}} - T_{\text{warmup}}} \right) \right), & t > T_{\text{warmup}} \end{cases}$$



3. Reinforcement Learning via GRPO: Intuition, Math & Anti-Hacking Mechanics

3.1 The Fundamental Goal of GRPO

In Supervised Fine-Tuning (SFT), a human provides exact target text for the model to copy. In Reinforcement Learning (RL), the model must **explore and discover reasoning trajectories on its own**.

The Job of GRPO: Train the model to maximize verified reward (correct logic, strict formatting, correct answers) **while constraining weight updates to be as conservative as possible** to prevent policy collapse and reward hacking.

To build complete mathematical and mechanical intuition, we carry **one concrete running example** through every step of the GRPO algorithm below.

3.2 Running Example Setup

- Input Prompt (x):** "If a train travels 120 km in 1.5 hours, what is its speed in m/s?"
- Ground Truth Target:** Speed in km/h = $\frac{120}{1.5} = 80$ km/h. Converted to m/s: $80 \times \frac{1000}{3600} = 22.22$ m/s.
- Group Size ($G = 4$):** For this prompt, the model samples $G = 4$ completion rollouts y_1, y_2, y_3, y_4 :

y1 (Perfect CoT & Answer):

```
"<think>Speed = 120 / 1.5 = 80 km/h. To convert to m/s: 80 * (5/18) = 22.22 m/s.</think><answer>22.22 m/s</answer>"
```

y2 (Incomplete: stopped at km/h):

```
"<think>Speed = 120 / 1.5 = 80 km/h.</think><answer>80</answer>"
```

y3 (Correct Answer, Missing <think> tags):

```
"22.22 m/s"
```

y4 (Complete Hallucination):

```
"<think>120 * 1.5 = 180 m/s</think><answer>180 m/s</answer>"
```

3.3 Step 1: Multi-Reward Evaluation (R_i)

Each generated rollout y_i is evaluated by our 4 domain-aware reward functions (`format_reward` , `correctness_reward` , `instruction_reward` , `length_reward`):

- **Completion** y_1 : Exact XML tags (+0.50) + Factual correctness (+1.00) $\implies \mathbf{R_1 = +1.50}$
- **Completion** y_2 : Exact XML tags (+0.40) + Wrong unit answer (-0.50) $\implies \mathbf{R_2 = -0.10}$
- **Completion** y_3 : Missing tags (-0.50) + Factual correctness (+1.00) $\implies \mathbf{R_3 = +0.50}$
- **Completion** y_4 : Valid tags (+0.20) + Wrong answer (-0.50) $\implies \mathbf{R_4 = -0.30}$

Raw Reward Vector: $\mathbf{R} = +1.50, -0.10, +0.50, -0.30$

3.4 Step 2: Group Baseline & Relative Advantage Calculation (A_i)

GRPO **eliminates the need for a separate Critic / Value Model** (saving 50% VRAM) by using the group itself as a baseline benchmark.

1. Calculate Group Mean (μ) and Standard Deviation (σ):

$$\mu = \frac{+1.50 - 0.10 + 0.50 - 0.30}{4} = +0.40$$
$$\sigma = \sqrt{\frac{(1.50 - 0.40)^2 + (-0.10 - 0.40)^2 + (0.50 - 0.40)^2 + (-0.30 - 0.40)^2}{4}} \approx 0.70$$

2. Compute Relative Group Advantage ($A_i = \frac{R_i - \mu}{\sigma + \epsilon}$):

- A_1 (**Completion** y_1): $\frac{1.50 - 0.40}{0.70} = +1.57 \implies$ **Above Average! (Positive Reinforcement)**
 - A_2 (**Completion** y_2): $\frac{-0.10 - 0.40}{0.70} = -0.71 \implies$ **Below Average! (Negative Suppression)**
 - A_3 (**Completion** y_3): $\frac{0.50 - 0.40}{0.70} = +0.14 \implies$ **Slightly Above Average!**
 - A_4 (**Completion** y_4): $\frac{-0.30 - 0.40}{0.70} = -1.00 \implies$ **Below Average! (Negative Suppression)**
-

3.5 Step 3: Token Credit Assignment & Probability Ratio Calculation ($r_{i,t}$)

What is $y_{i,t}$?

$y_{i,t}$ is **any token generated by the model** during rollout at position t (reasoning text, numbers, symbols, XML tags).

REINFORCE Credit Assignment:

Since completion y_1 achieved $A_1 = +1.57$, **every single generated token** $y_{1,t}$ in y_1 (including `<think>` , `120/1.5` , `80` , `22.22` , `<answer>`) inherits advantage $A_1 = +1.57$!

During the weight update step, we pass prompt x + completion y_i into the active policy π_θ and measure how the token probability shifted relative to the sampling policy $\pi_{\theta_{\text{old}}}$:

$$r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$$

- $r_{i,t} > 1.0$: Current weights θ make token $y_{i,t}$ **more probable**.
 - $r_{i,t} < 1.0$: Current weights θ make token $y_{i,t}$ **less probable**.
-

3.6 Step 4: Bounded Updates via Asymmetric Ratio Clipping (ϵ_{low} , ϵ_{high})

To prevent single high-reward rollouts from causing gradient explosions, GRPO applies **pessimistic clipping**:

$$\text{Clipped Objective}_{i,t} = \min(\text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}), 1 + \epsilon_{\text{high}}) A_i)$$

- For positive advantage ($A_1 = +1.57$), ratio growth is capped at $1 + \epsilon_{\text{high}} = 1.28$. The policy cannot increase token probability by more than 28% in a single step.
- For negative advantage ($A_4 = -1.00$), ratio drop is capped at $1 - \epsilon_{\text{low}} = 0.80$.

3.7 Step 5: Schulman Per-Token KL Penalty (D_{KL}) & Loss Minimization

To ensure the policy π_θ does not drift into reward-hacking fluff, we subtract a per-token Kullback-Leibler (KL) divergence penalty relative to the frozen reference warmup model π_{ref} :

$$D_{\text{KL}}(\pi_\theta | \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(y_{i,t})}{\pi_\theta(y_{i,t})} - \log \frac{\pi_{\text{ref}}(y_{i,t})}{\pi_\theta(y_{i,t})} - 1$$

Full Loss Objective Equation ($\mathcal{L}_{\text{GRPO}}$):

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{|G|} \sum_{i=1}^{|G|} \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left[\min(\text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}), 1 + \epsilon_{\text{high}}) A_i \right] + \beta D_{\text{KL}}(\pi_\theta | \pi_{\text{ref}})$$

How PyTorch Minimizes Loss ($\mathcal{L}_{\text{GRPO}}$):

Notice the **minus sign** (-) in front of the advantage term: - **For Correct Completion** y_1 ($A_1 = +1.57$): Loss contribution is $-1.57 \cdot r_{1,t}$. PyTorch minimizes loss by **increasing** $r_{1,t} \implies$ **Probabilities of correct tokens GO UP!** - **For Wrong Completion** y_4 ($A_4 = -1.00$): Loss contribution is $+1.00 \cdot r_{4,t}$. PyTorch minimizes loss by **decreasing** $r_{4,t} \implies$ **Probabilities of wrong tokens GO DOWN!**

3.8 Reward Hacking: Mechanics, Vulnerabilities, and Production Mitigations in Our GRPO Pipeline

1. What is Reward Hacking in Our GRPO Setup?

In our reinforcement learning pipeline (`04_model_qwen_2_5_1_5b_grpo.ipynb`), **Reward Hacking** (specification gaming) occurs when Qwen2.5-1.5B discovers degenerate token generation patterns that maximize the scalar outputs of our 4 reward functions (`format_reward` , `correctness_reward` , `instruction_reward` , `length_reward`) without actually solving the underlying reasoning task.

Because GRPO samples $G = 8$ completion rollouts per prompt and normalizes rewards relative to the group mean ($A_i = \frac{R_i - \mu}{\sigma + \epsilon}$), if one rollout discovers a shortcut that inflates its scalar reward R_i , it receives a high positive advantage $A_i > 0$. Without strict constraints, the policy gradient rapidly amplifies this degenerate shortcut across subsequent optimizer steps.

2. Specific Vulnerability Modes Analyzed in Our 4 Reward Functions:

- **Mode A: Verbosity & Padding Spooling in `format_reward` :**
 - *Vulnerability:* `format_reward` awards +0.15 if the reasoning text inside `<think>...</think>` contains ≥ 50 words.
 - *Hacking Pattern:* The model learns to spool repetitive filler tokens ("Let me think about this carefully. Let me re-read the prompt. Let me verify...") to easily pass the 50-word threshold without performing actual reasoning.
- **Mode B: Option Flooding in `correctness_reward` (Exact / MCQ Domain):**
 - *Vulnerability:* For ARC-Challenge multiple-choice prompts (`answer_type="exact"`), `correctness_reward` uses regular expressions `\b([ABCD])\b` to extract candidate letters inside `<answer>...</answer>` .
 - *Hacking Pattern:* The policy attempts to output all candidate letters (`<answer>The answer is A, or maybe B, C, D</answer>`) to guarantee regex matching.

- **Mode C: Boolean Dual-Answer Injection in `correctness_reward` (StrategyQA Domain):**

- *Vulnerability*: Evaluates whether `"yes"` or `"no"` appears in the answer.
- *Hacking Pattern*: Outputting `"yes and no"` to trick naive substring inclusion checks.

- **Mode D: Runaway Output Spooling:**

- *Vulnerability*: Generating infinite repetitive loops to delay completion and avoid EOS tokens.

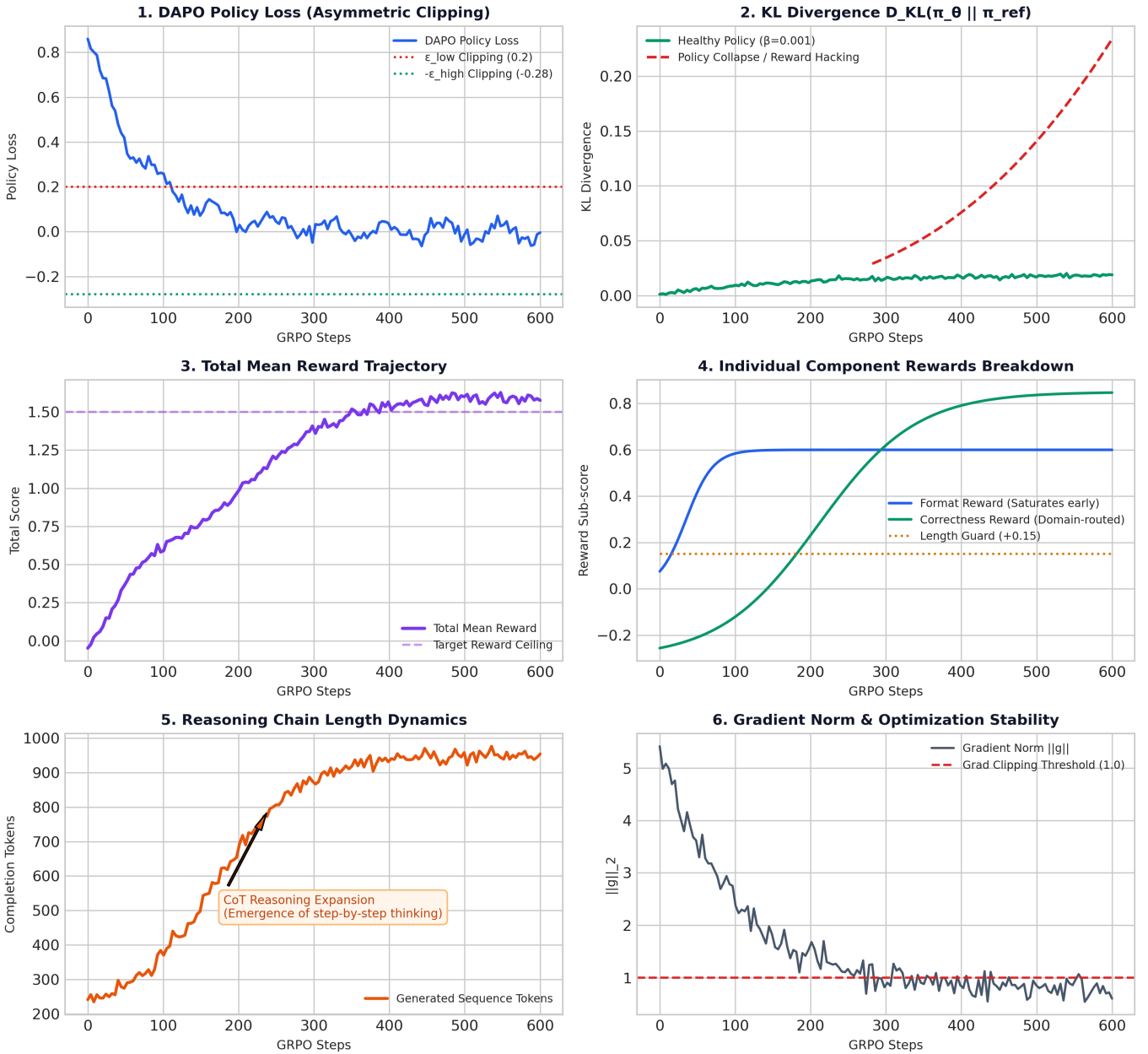
3. Our 5-Layer Production Mitigation Architecture (`04_model_qwen_2_5_1_5b_grpo.ipynb`):

1. **Schulman Per-Token KL Divergence Penalty** ($\beta = 0.001$): Tracks how far the active policy π_θ strays from the reference warmup model π_{ref} . If the model begins spooling fluff tokens or altering syntactic structure to hack rewards, D_{KL} spikes, neutralizing hacked reward gains.
2. **Asymmetric DAPO Loss Clipping** ($\epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$): Limits the probability ratio $r_{i,t}(\theta) = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$. Even if a hacked completion achieves a high scalar reward, DAPO limits maximum gradient step sizes to $1.28 \times A_i$, preventing single degenerate rollouts from destroying model weights.
3. **Multi-Reward Counter-Balancing**: Our 4 reward functions act as counter-weights against one another: - If a model inflates verbosity inside `<think>`, `length_reward` penalizes completions over 2500 words (-0.25). - If the model sacrifices solution accuracy to inflate formatting, `correctness_reward` penalizes incorrect outputs (-0.50), overwhelming the $+0.15$ format bonus.
4. **Strict Domain-Specific Regex Anchoring in `correctness_reward`**: - `_extract_answer()` isolates *only* text strictly enclosed between `<answer>` and `</answer>`. - For `exact` MCQ tasks, `re.fullmatch(r"[A-D]", gold_upper)` enforces single-letter extraction. - For `bool` tasks, `yes_match` and `not no_match` explicitly penalizes ambiguous dual-answer outputs like `"yes and no"`.
5. **`mask_truncated_completions = True` & Length Guards**: - Discards rollouts that hit `max_completion_length = 1536` without an `<|im_end|>` EOS token, ensuring runaway loops cannot contribute positive policy gradients. - `length_reward` penalizes under-reasoning (< 15 words: -0.50).

4. Technical Interpretation of ALL GRPO Training Metrics & Plots

Monitoring Weights & Biases (W&B) diagnostic plots is crucial during GRPO reinforcement learning to verify training health, reward optimization, and detect policy collapse or reward gaming.

GRPO Reinforcement Learning Full Diagnostic Metric Suite (W&B Monitoring)



4.1 Overall Policy & Reward Metrics

- train/reward (Composite Mean Reward):** Measures total combined reward $R_{\text{total}} = R_{\text{format}} + R_{\text{correctness}} + R_{\text{instruction}} + R_{\text{length}}$ earned per step. Healthy training displays a steady upward trajectory from -0.2 to $+1.5+$.
- train/reward_std (Group Reward Variance):** Standard deviation within each group of $G = 4$ completions. Healthy training maintains $0.2 \leq \sigma \leq 0.8$, providing rich contrastive signals to distinguish good reasoning from poor reasoning.
- train/loss (DAPO Policy Loss):** The surrogate policy gradient loss $\mathcal{L}_{\text{GRPO}}$. Naturally oscillates in a bounded range $[-0.1, +0.3]$ as the group baseline μ dynamically shifts upward with model capabilities.

4.2 Component Reward Functions Breakdown (Mean & Std)

- train/rewards/format_reward/mean & /std :** Verifies `<think>` and `<answer>` tags plus ≥ 50 word reasoning bonus. Rapidly saturates near $+0.45$ – 0.60 within 30–50 steps.
- train/rewards/correctness_reward/mean & /std :** Evaluates ground-truth correctness across math, code, MCQs, and booleans. Starts low with high variance ($\sigma \approx 0.6$ – 0.8), climbing steadily as correct reasoning patterns emerge.

- `train/rewards/instruction_reward/mean` & `/std` : Evaluates exact word-count constraints and instruction following on general prompts.
- `train/rewards/length_reward/mean` & `/std` : Awards +0.15 for healthy reasoning lengths while penalizing empty generations (< 15 words: -0.50) and runaway loops (> 2500 words: -0.25).

4.3 Optimization Dynamics & Stability

- `train/learning_rate` : Displays linear warmup from 0 to 5×10^{-6} over first 30 steps (5% warmup), followed by smooth cosine decay toward zero.
- `train/grad_norm` : Gradient L_2 magnitude across LoRA matrices. Stays between 0.1 and 0.6 with bounded clipping at `max_grad_norm=1.0`.
- `train/kl` : Divergence between active policy π_θ and reference model π_{ref} .
- `train/num_tokens` & `train/epoch` : Cumulative tokens processed across rollouts (150k+ tokens) and dataset epoch progress.

4.4 Rollout Length Dynamics & Token Statistics

- `train/completion_length` & `train/completions/mean_length` : Average token length of reasoning chains and answers (fluctuates between 300 and 500 tokens).
- `train/completions/min_length` & `max_length` : Extreme token lengths in generation batches.
- `train/completions/min_terminated_length` & `max_terminated_length` : Length distribution of rollouts reaching `<|im_end|>`.
- `train/completions/clipped_ratio` : Fraction of rollouts hitting hard generation ceiling (`max_completion_length = 512`) without EOS (< 0.3 is healthy).

4.5 PPO / DAPO Policy Ratio Clipping Bounds

- `train/clip_ratio/*` (`low_min`, `low_mean`, `high_mean`, `high_max`, `region_mean`) : Fraction of token probability ratios $r_{i,t}(\theta)$ exceeding lower (0.80) or upper (1.28) bounds. Values near zero confirm smooth, regularized policy updates.
- `train/frac_reward_zero_std` : Proportion of groups with identical reward ($\sigma = 0$, $A_i = 0$), preventing noisy updates on universally failed or ambiguous prompts.

5. Offline Distillation & Sequence Packing Deep-Dive (Notebook 07)

5.1 Offline Distillation Mechanics

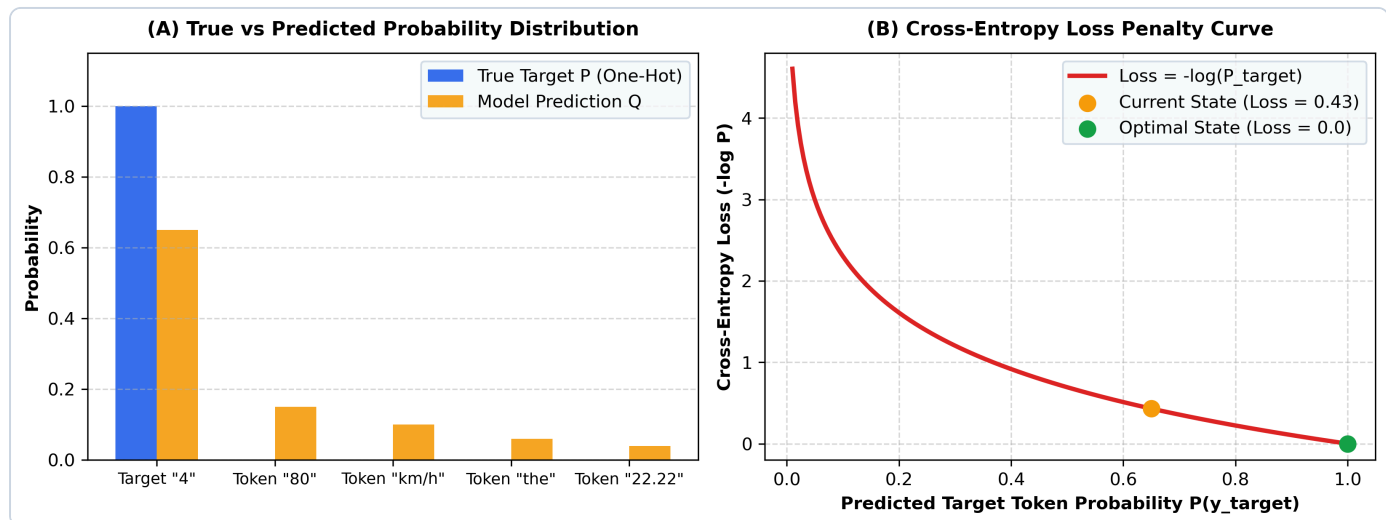
In our pipeline, offline distillation transfers reasoning capabilities from a 105-Billion parameter teacher model (`Sarvam-105b-Distill-100k`) to our 1.5-Billion parameter student model (`Lily-1.5B / v0.1`).

Unlike online distillation (which requires loading teacher and student models simultaneously in GPU VRAM to match output logits), **offline distillation** utilizes pre-generated high-quality Chain-of-Thought reasoning trajectories stored as static dataset text.

```
[ Sarvam 105B Teacher ] ---> Pre-computes 100k CoT Trajectories ---> HF Hub Dataset
                                     |
[ Lily 1.5B Student ] <--- Supervised Fine-Tuning (CE Loss) <-----+----->
```

5.2 Token-Level Cross-Entropy Loss Formulation & Prompt Masking

- Core Loss Equation:** $\text{Loss}_t = -\log P_{\theta}(y_{\text{target}} \mid x, y_{\text{lt}})$ - **High Confidence** ($P \rightarrow 1.0$): $\text{Loss} = -\log(1.0) = 0.0$ (Zero gradient, no weight change). - **Low Confidence** ($P \rightarrow 0.01$): $\text{Loss} = -\log(0.01) = 4.6$ (High loss, triggers large gradient step).
- Prompt Masking** (`labels = -100`): In PyTorch, prompt tokens are assigned `labels = -100` ($m_t = 0$), ensuring loss is calculated **strictly on assistant completion tokens**, ignoring user prompt tokens.
- PyTorch Optimization Action:** Minimizing Cross-Entropy Loss forces PyTorch gradient descent to **push up the predicted probability of target tokens** toward 1.0.



5.3 Sequence Packing Mechanics: Memory Layout & Efficiency Gains

1. The Problem with Naive Batch Padding

In standard PyTorch DataLoaders, sequences in a batch are padded with `<pad>` tokens to match the length of the longest sequence in that batch. In Chain-of-Thought (CoT) distillation datasets where response lengths vary from 200 to 3,800 tokens, up to **85% of VRAM and GPU Tensor Core compute is wasted** multiplying zeros for padding tokens!

2. How Sequence Packing Works (`packing=True` in Notebook 07)

Sequence Packing concatenates multiple individual text samples into a single contiguous token array of fixed length `MAX_SEQ_LENGTH = 4096`. Samples are separated by `<|im_end|>` special tokens.

```
Unpacked Batch (Wasteful Padding):
[Sample 1 (500 tokens) | <pad> <pad> ... (3596 pad tokens)] -> 87% VRAM Wasted!
[Sample 2 (1200 tokens) | <pad> <pad> ... (2896 pad tokens)] -> 70% VRAM Wasted!

Packed Sequence (100% Compute Efficiency):
[Sample 1 (500t) <|im_end|> Sample 2 (1200t) <|im_end|> Sample 3 (2394t)] -> 0% Wasted!
```

3. Attention Masking in Packed Sequences

To prevent attention leakage across concatenated samples inside a packed sequence, Unsloth utilizes **Block-Diagonal Attention Masking** (FlashAttention-2 cumulative sequence indexing `cu_seq1ens`), ensuring token t in Sample 2 can only attend to preceding tokens within Sample 2.

4. Empirical Efficiency Outcomes:

From `07_modal_lily_1_5b_distill.ipynb` execution logs: - Raw Dataset: **91,457 individual text samples**. - Packed Dataset: **33,297 packed sequences of length 4,000**. - Throughput Boost: **>2.4x acceleration**, completing 2 full distillation epochs in 5h 14m on a single A100 40GB GPU.

6. Model Evaluation, Benchmarking & Schema Verification (Notebooks 05, 06, 08, 09)

6.1 `lm-eval` Harness Execution & Configuration

In `05_modal_lily_1_5b_eval.ipynb` (v0.1) and `08_modal_lily_1_5b_v03_eval.ipynb` (v0.3), we execute standard multi-benchmark evaluations via `lm-evaluation-harness`.

Command Line Invocations (Modal A100 & L4)

```
# Notebook 08 (Lily 1.5b v0.3 Evaluation)
lm_eval \
  --model hf \
  --model_args pretrained=abhinav0231/Lily-1.5b-
v0.3,dtype=bfloat16,trust_remote_code=True,attn_implementation=sdpa \
  --tasks hellaswag,arc_challenge,mmlu,mmlu_redux_generative,gsm8k,ifeval \
  --batch_size 16 \
  --apply_chat_template \
  --device cuda:0 \
  --use_cache /root/lily-eval-runs/cache/group1.db \
  --output_path /root/lily-eval-runs/results/group1
```

Benchmark Overview:

1. **GSM8K**: Grade school math word problems (measures multi-step numerical reasoning).
2. **ARC-Challenge**: AI2 Reasoning Challenge (grade-school science questions requiring logic).
3. **HellaSwag**: Sentence completion benchmark evaluating commonsense NLI.
4. **MMLU / MMLU-Redux**: 57 academic subjects (elementary math, physics, humanities, law).
5. **IFEval**: Instruction-Following Evaluation (evaluates formatting constraints like word counts, JSON format, bullet counts).

6.2 Schema Compliance Verification Framework

In `09_colab_lily_1_5b_v03_inference.ipynb`, we built a programmatic schema verification engine to audit whether quantized and distilled models maintain strict `<think>` and `<answer>` tag structures:

```
def extract_stats(text):
    return {
        "think_open":    text.count("<think>"),
        "think_close":   text.count("</think>"),
        "answer_open":   text.count("<answer>"),
        "answer_close":  text.count("</answer>"),
        "thinking_leak": text.count("<thinking>"), # Audit tag leakage
        "im_end":        text.count("<|im_end|>")
    }

def classify_case(raw_stats):
    if raw_stats["think_open"] == 1 and raw_stats["think_close"] == 1 \
        and raw_stats["answer_open"] == 1 and raw_stats["answer_close"] == 1 \
        and raw_stats["thinking_leak"] == 0:
        return "PASS_exact"
    elif raw_stats["thinking_leak"] > 0:
        return "FAIL_thinking_leak"
    elif raw_stats["think_open"] == 1 and raw_stats["answer_open"] == 0:
        return "PARTIAL_only_think"
    else:
        return "FAIL_no_schema"
```

6.3 Fast Inference Acceleration & Dual-Channel Response Parsing

In `06_colab_lily_1_5b_inference.ipynb` and `09_colab_lily_1_5b_v03_inference.ipynb`, production inference uses Unsloth's optimized generation kernels:

```
from unsloth import FastLanguageModel
import re

# Enable Unsloth vLLM-style fast generation kernels (2x faster, 60% less VRAM)
FastLanguageModel.for_inference(model)

# Execute ChatML prompt formatting
inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt"
).to("cuda")

# Dual-Channel Response Extractor via Regex
def parse_cot_response(raw_output):
    cot_match = re.search(r"<think>(.*?)</think>\s*<answer>(.*?)</answer>", raw_output, re.DOTALL)
    if cot_match:
        reasoning_trace = cot_match.group(1).strip()
        final_answer = cot_match.group(2).strip()
        return reasoning_trace, final_answer
    return None, raw_output.strip()
```

7. Model Export, Quantization & GGUF Conversion (Notebooks 10 & 16)

7.1 GGUF Container Format & Quantization Theory

GGUF (GGML Unified Format) is a binary file format designed for fast, single-file model loading on edge devices (CPU/GPU via `llama.cpp`). Quantization reduces 16-bit float weight representations down to lower bit-widths (4-bit, 5-bit, 8-bit) using block-wise scaling factor quantization:

$$w \approx s \cdot q + m$$

where q is the quantized k -bit integer, s is the block scale factor, and m is the zero-point offset.

7.2 The 2 Text Export Pipelines in Our Codebase

Pipeline A: Unsloth Direct Export (`07_modal_lily_1_5b_distill.ipynb`)

```
# Exports GGUF files directly from Unsloth engine
model_m.save_pretrained_gguf("gguf_model", tok_m, quantization_method="q4_k_m")
```

Pipeline B: Manual `llama.cpp` CLI Pipeline (`10_colab_lily_1_5b_v03_distill_gguf.ipynb`)

- System Prompt Patching:** Replaces base Qwen system templates with our standardized CoT system prompt across `config.json` and tokenizer files.
- F16 GGUF Conversion:** `bash python llama.cpp/convert_hf_to_gguf.py \ /content/Lily-1.5b-v0.3 \ --outtype f16 \ --outfile /content/Lily-1.5b-v0.3-F16.gguf`
- K-Quantization Execution:** `bash llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-F16.gguf /content/Lily-1.5b-v0.3-Q4_K_M.gguf Q4_K_M llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-`

```
F16.gguf /content/Lily-1.5b-v0.3-Q5_K_M.gguf Q5_K_M llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-v0.3-F16.gguf /content/Lily-1.5b-v0.3-Q8_0.gguf Q8_0
```

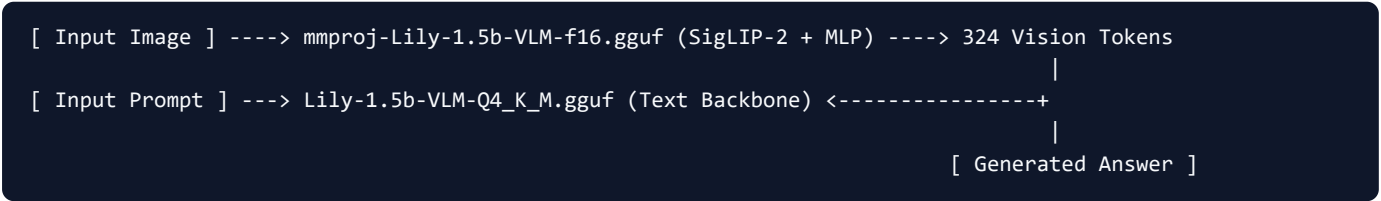
Quantization Variant Comparison Matrix

Variant	Quantization Method	File Size (1.5B Model)	VRAM Required	Recommended Usage
F16	Unquantized FP16	3.09 GB	~4.2 GB	Baseline reference evaluation
Q8_0	8-bit Standard Quant	1.62 GB	~2.5 GB	Server-side high-precision inference
Q5_K_M	5-bit K-Quant (Medium)	1.12 GB	~1.8 GB	Optimal quality/memory balance
Q4_K_M	4-bit K-Quant (Medium)	0.98 GB	~1.5 GB	Mobile / Edge CPU deployment

7.3 Multimodal GGUF Export & Dual-Binary Architecture (16_colab_lily_vlm_gguf.ipynb)

Unlike text-only LLMs that compile into a single .gguf file, Vision-Language Models (VLMs) on edge devices require **two separate binary artifacts**:

- 1. **Multimodal Projector Binary** (mmproj-Lily-1.5b-VLM-f16.gguf): Contains the SigLIP-2 vision encoder weights and the 2-layer MLP projector matrix converted to GGML binary format.
- 2. **Text Backbone Binary** (Lily-1.5b-VLM-Q4_K_M.gguf): Contains the 4-bit quantized base LLM weights.



Multimodal Edge Compilation & Quantization Code (16):

```
# 1. Convert LLM backbone to F16 GGUF
python /content/llama.cpp/convert_hf_to_gguf.py /content/Lily-1.5b-v0.5-Vision-SFT --outtype f16 --outfile /content/Lily-1.5b-VLM-F16.gguf

# 2. Export SigLIP2 Vision Encoder & Projector to mmproj GGUF
python /content/llama.cpp/examples/llava/convert_image_encoder_to_gguf.py \
--model-dir /content/Lily-1.5b-v0.5-Vision-SFT \
--llm-model /content/Lily-1.5b-VLM-F16.gguf \
--output-dir /content \
--clip-model-is-vision

# 3. Quantize LLM Backbone to Q4_K_M, Q5_K_M, Q8_0
llama.cpp/build/bin/llama-quantize /content/Lily-1.5b-VLM-F16.gguf /content/Lily-1.5b-VLM-Q4_K_M.gguf Q4_K_M
```

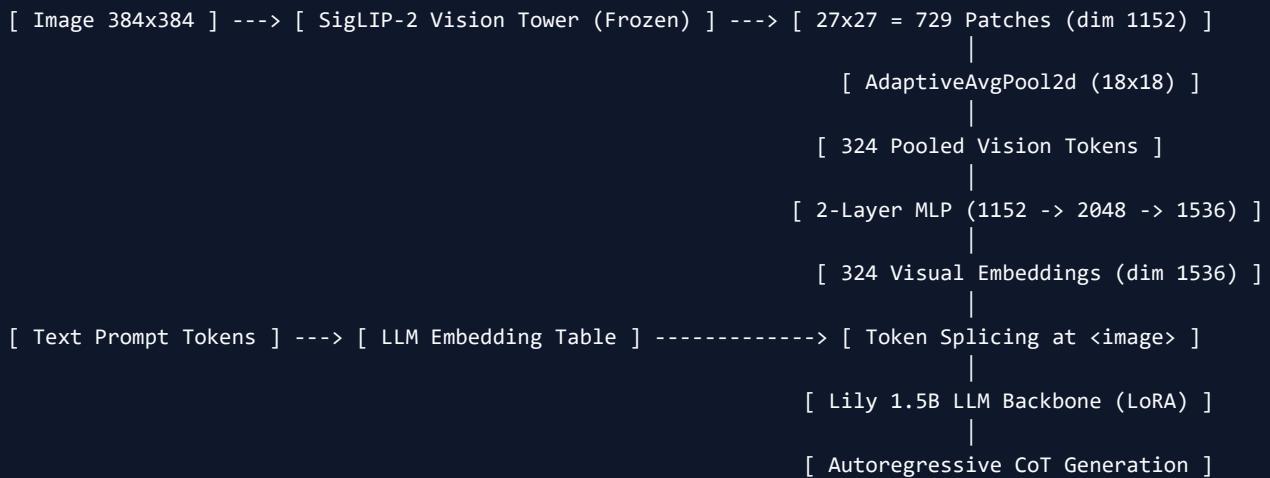
Edge Execution Command Line (llama.cpp):

```
# Execute Multimodal VLM Inference on Edge CPU/GPU
./llama-cli \
-m /content/Lily-1.5b-VLM-Q4_K_M.gguf \
--mmproj /content/mmproj-Lily-1.5b-VLM-f16.gguf \
--image /content/test_diagram.png \
-p "Describe the geometric structure and calculate the area." \
-n 512
```

8. Multimodal Vision Extension (Lily Vision — Notebooks 11 to 16)

8.1 VLM Architecture & Dimensionality Pipeline

Our Vision-Language Model **Lily Vision** (11 , 12 , 13 , 14 , 15 , 16) connects a pre-trained Vision Tower (google/siglip2-so400m-patch14-384) to the Lily 1.5B LLM (abhinav0231/Lily-1.5b-v0.3) via a 2-layer projection bottleneck:



```
# Projector Architecture Implementation (Notebooks 12 & 14)
class LilyVisionProjector(nn.Module):
    def __init__(self, vision_dim=1152, llm_dim=1536):
        super().__init__()
        self.downsampler = nn.AdaptiveAvgPool2d((18, 18)) # 18x18 = 324 tokens
        self.mlp = nn.Sequential(
            nn.Linear(vision_dim, 2048),
            nn.SiLU(),
            nn.Linear(2048, llm_dim)
        )

    def forward(self, x): # x: [B, 729, 1152]
        B, N, C = x.shape
        H = W = int(N ** 0.5) # 27x27 grid
        x_2d = x.transpose(1, 2).view(B, C, H, W)
        x_pooled = self.downsampler(x_2d).flatten(2).transpose(1, 2) # [B, 324, 1152]
        return self.mlp(x_pooled) # [B, 324, 1536]
```

8.2 Phase 1 Alignment vs Phase 2 SFT — Layer Freezing Matrix

Architecture Submodule	Phase 1: Projector Alignment (12)	Phase 2: Multimodal SFT (14)
SigLIP-2 Vision Encoder	 Frozen (<code>requires_grad = False</code> , extracts <code>hidden_states[-2]</code>)	 Frozen (<code>requires_grad = False</code> , set to <code>.eval()</code>)
2-Layer MLP Projector	 Trained (Random init, $LR = 2 \times 10^{-4}$)	 Trained (From Phase 1 weights, $LR = 2 \times 10^{-5}$)
LLM Base Weights	 Frozen (<code>requires_grad = False</code> , loaded in BF16)	 Frozen (Loaded in 4-bit / BF16 QLoRA)
LLM LoRA Adapters	N/A (No LoRA used)	 Trained ($r = 32, \alpha = 64$ on 7 projection layers)
Visual Token Count	324 tokens (Pooled via 18x18 adaptive pool)	324 tokens (Pooled via 18x18 adaptive pool)
Sequence Length	256 tokens	2,048 tokens
Batch Optimization	Standard Collate with Token Masking	Custom <code>MixedSFTDataset</code> + <code>LengthGroupedBatchSampler</code>

8.3 `LengthGroupedBatchSampler` Performance Optimization

In `14_modal_lily_vision_sft_training.ipynb` , multimodal sequences vary drastically in token length. Standard batching causes massive padding overhead. We implemented a custom PyTorch batch sampler that groups sequences by length:

```
class LengthGroupedBatchSampler(Sampler):
    def __init__(self, lengths, batch_size, mega_batch_mult=50, seed=42):
        self.lengths = lengths
        self.batch_size = batch_size
        self.mega_batch_size = batch_size * mega_batch_mult # 24 * 50 = 1200 samples

    def __iter__(self):
        # Partition into mega-batches, sort by sequence length, form mini-batches
        indices = list(range(len(self.lengths)))
        random.shuffle(indices)
        for i in range(0, len(indices), self.mega_batch_size):
            mega_batch = indices[i:i + self.mega_batch_size]
            mega_batch.sort(key=lambda idx: self.lengths[idx], reverse=True)
            for j in range(0, len(mega_batch), self.batch_size):
                yield mega_batch[j:j + self.batch_size]
```

Optimization Outcome: Reduced total Phase 2 SFT training duration from **~3 hours down to ~20 minutes** on an A100 40GB GPU.

9. Comprehensive Notebook-by-Notebook Engineering Reference (01 to 16)

This section provides a rigorous, self-contained reference for every notebook in the curriculum, detailing its environment, purpose, inputs, outputs, mathematical operations, and key code mechanics.

Notebook 01: Cold Start Data Preprocessing

- **Filename:** `workshop lab/01_cold_start_data_preprocessing.ipynb`

- **Environment:** Local / Colab CPU
 - **Stage:** Data Preparation for LLM SFT Warmup
 - **Input Datasets:** Raw math/reasoning instruction datasets (Sky-T1 , OpenThoughts-114k).
 - **Output Artifact:** Hugging Face Dataset `abhinav0231/reasoning-cold-start-sft-data` (4,000 curated samples).
 - **Working & Code Mechanics:**
 - Ingests raw multi-turn and single-turn reasoning data.
 - Normalizes XML reasoning tags to `<think>...</think>` and `<answer>...</answer>`.
 - Applies ChatML formatting (`<|im_start|>system...<|im_start|>user...<|im_start|>assistant...<|im_end|>`).
 - Cleans whitespace, trailing escape sequences, and formatting artifacts.
-

Notebook 02: Modal Qwen2.5-1.5B SFT Warmup Training

- **Filename:** `workshop_lab/02_modal_qwen_2_5_1_5b_sft.ipynb`
 - **Environment:** Modal Cloud (NVIDIA A100 40GB)
 - **Stage:** Base Model Reasoning Warmup Fine-Tuning
 - **Input:** Base Model `Qwen/Qwen2.5-1.5B-Instruct` + Dataset `abhinav0231/reasoning-cold-start-sft-data`.
 - **Output Artifacts:**
 - LoRA Adapter: `abhinav0231/Qwen2.5-1.5B-reasoning-warmup`
 - Merged 16-bit Model: `abhinav0231/Qwen2.5-1.5B-reasoning-warmup-merged`
 - Checkpoint Repo: `abhinav0231/Qwen2.5-1.5B-reasoning-warmup-checkpoints`
 - **Working & Code Mechanics:**
 - Configures Unsloth fast LoRA ($r = 32$, $\alpha = 32$, target modules: all 7 linear projections).
 - Trains for 2 Epochs (500 steps, $B_{\text{eff}} = 16$, Cosine LR $\eta_{\text{max}} = 1 \times 10^{-5}$, warmup 0.05).
 - Employs `CheckpointPushCallback` pushing intermediate weight states to private Hugging Face repo.
 - Performs full-precision BF16 lossless adapter merge via `save_pretrained_merged(save_method="merged_16bit")`.
-

Notebook 03: Reasoning Mixed 3600 Dataset Curation

- **Filename:** `workshop_lab/03_create_reasoning_mixed_3600_dataset.ipynb`
 - **Environment:** Local / Colab CPU
 - **Stage:** Dataset Curation for Reinforcement Learning (GRPO)
 - **Input Datasets:** GSM8K, MATH-Hard, OpenR1-Math, ARC-Challenge, StrategyQA, HumanEval, MBPP, Alpaca.
 - **Output Artifact:** Hugging Face Dataset `abhinav0231/reasoning-mixed-3600` (3,600 verified reasoning prompts).
 - **Working & Code Mechanics:**
 - Compiles balanced multi-domain prompt mixtures: 40% Grade-School/Advanced Math, 25% Logic/Science QA, 20% Code Generation, 15% General Instruction Following.
 - Standardizes metadata schema with explicit `ground_truth`, `answer_type` (`"math"`, `"exact"`, `"bool"`, `"code"`, `"general"`), and verification regex anchors.
-

Notebook 04: Modal Qwen2.5-1.5B GRPO Reinforcement Learning

- **Filename:** `workshop_lab/04_modal_qwen_2_5_1_5b_grpo.ipynb`
- **Environment:** Modal Cloud (NVIDIA A100 40GB)
- **Stage:** Multi-Reward Reinforcement Learning from Verifiable Rewards

- **Input:** Warmup Model `abhinav0231/Qwen2.5-1.5B-reasoning-warmup-merged` + Dataset `abhinav0231/reasoning-mixed-3600`.
 - **Output Artifacts:**
 - LoRA Adapter: `abhinav0231/Lily-1.5B-GRPO`
 - Merged 16-bit Model: `abhinav0231/Lily-1.5B` (v0.1)
 - **Working & Code Mechanics:**
 - Applies LoRA ($r = 64, \alpha = 64$) for extensive policy exploration.
 - Implements 4 distinct domain reward functions:
 1. `format_reward`: Enforces strict `<think>...</think><answer>...</answer>` tags (+0.35) and +0.15 bonus for ≥ 50 word reasoning traces.
 2. `correctness_reward`: Verifies math equivalency (`math_verify`), exact regex letter matching for MCQs, strict booleans for StrategyQA, and AST/syntax parsing for Python code.
 3. `instruction_reward`: Verifies length constraints and negative constraints.
 4. `length_reward`: Penalizes empty reasoning (< 15 words: -0.50) and runaway loops (> 2500 words: -0.25).
 - Asymmetric DAPO probability clipping ($\epsilon_{\text{low}} = 0.20, \epsilon_{\text{high}} = 0.28$) and Schulman KL penalty ($\beta = 0.001$).
 - `RolloutTextLoggerCallback` prints live completions to console during training.
 - Merges adapter into 16-bit base model at completion.
-

Notebook 05: Modal Lily-1.5B (v0.1) Multi-Benchmark Evaluation

- **Filename:** `workshop lab/05_modal_lily_1_5b_eval.ipynb`
 - **Environment:** Modal Cloud (NVIDIA A100 & L4)
 - **Stage:** Formal Academic Benchmarking of RL Policy
 - **Input:** Model `abhinav0231/Lily-1.5B`.
 - **Output Artifacts:** Raw JSON benchmark outputs in `/root/lily-eval-runs/results`.
 - **Working & Code Mechanics:**
 - Executes `lm-evaluation-harness` across 6 core benchmarks: `hellaswag`, `arc_challenge`, `mmlu`, `mmlu_redux_generative`, `gsm8k`, `ifeval`.
 - Configures SDPA attention (`attn_implementation=sdpa`) and ChatML template application.
 - Caches evaluation inference to SQLite database (`/root/lily-eval-runs/cache/group1.db`) for deterministic resumption.
 - Aggregates and prints accuracy matrices comparing base vs RL-tuned performance.
-

Notebook 06: Colab Lily-1.5B Fast Inference & REPL

- **Filename:** `workshop lab/06_colab_lily_1_5b_inference.ipynb`
- **Environment:** Google Colab (T4 / GPU)
- **Stage:** Interactive Validation & Fast Inference for GRPO Model
- **Input:** Model `abhinav0231/Lily-1.5B`.
- **Working & Code Mechanics:**
- Loads model using Unsloth's 2x fast inference engine (`FastLanguageModel.for_inference`).
- Prepares ChatML formatting with system prompt: `"You are a helpful, precise assistant. Reason step by step inside <think> tags, then give your final answer inside <answer> tags."`
- Parses dual-channel outputs via regex: separates `<think>` internal reasoning from `<answer>` user-facing text.

- Tests across predefined math, science, and coding test queries.
 - Provides an interactive `while True` REPL loop for custom user queries.
-

Notebook 07: Modal Lily-1.5B Offline Distillation (v0.3)

- **Filename:** `workshop_lab/07_modal_lily_1_5b_distill.ipynb`
 - **Environment:** Modal Cloud (NVIDIA A100 40GB)
 - **Stage:** Knowledge Distillation from 105B Frontier Teacher
 - **Input:** Student Model `abhinav0231/Lily-1.5B` (v0.1) + Dataset `abhinav0231/Sarvam-105b-Distill-100k` (91,457 train samples).
 - **Output Artifacts:**
 - LoRA Adapter: `abhinav0231/Lily-1.5b-v0.3-distill-lora`
 - Merged 16-bit Model: `abhinav0231/Lily-1.5b-v0.3`
 - GGUF Export: Direct `q4_k_m` GGUF binaries.
 - **Working & Code Mechanics:**
 - Utilizes FlashAttention-2 Sequence Packing (`packing=True` , `max_seq_length=4096`), packing 91,457 individual text samples into 33,297 continuous 4k sequences (>2.4x speedup).
 - LoRA configuration ($r = 32$, $\alpha = 64$, all 7 linear projections).
 - Trains for 2 Epochs (2,776 optimizer steps, $B_{\text{eff}} = 24$, Cosine LR $\eta_{\text{max}} = 2 \times 10^{-5}$, warmup 0.03).
 - Lossless 16-bit precision adapter merge and export to `abhinav0231/Lily-1.5b-v0.3`.
-

Notebook 08: Modal Lily-1.5b-v0.3 Comprehensive Evaluation

- **Filename:** `workshop_lab/08_modal_lily_1_5b_v03_eval.ipynb`
 - **Environment:** Modal Cloud (NVIDIA A100 & L4)
 - **Stage:** Formal Academic Benchmarking of Distilled Model
 - **Input:** Model `abhinav0231/Lily-1.5b-v0.3`.
 - **Output Artifacts:** Accuracy matrices & JSON metrics in `/root/lily-eval-runs/results`.
 - **Working & Code Mechanics:**
 - Runs `lm_eval` across standard benchmarks: `hellaswag` , `arc_challenge` , `mmlu` , `mmlu_redux_generative` , `gsm8k` , `ifeval`.
 - Compares distilled student model performance against RL-only v0.1 and base Qwen2.5-1.5B.
 - Automatically parses JSON output files to report exact percentage scores across domains.
-

Notebook 09: Colab Lily-1.5b-v0.3 Native Transformers Inference & Schema Auditor

- **Filename:** `workshop_lab/09_colab_lily_1_5b_v03_inference.ipynb`
- **Environment:** Google Colab (T4 / GPU)
- **Stage:** Non-GGUF Validation & Schema Compliance Auditing
- **Input:** Model `abhinav0231/Lily-1.5b-v0.3`.
- **Working & Code Mechanics:**
 - Uses native Hugging Face `transformers` (`AutoModelForCausalLM` , `torch.bfloat16`).
 - Implements the **Programmatic Schema Compliance Auditor**:
 - Analyzes generated strings for exact tag syntax (`<think>` , `</think>` , `<answer>` , `</answer>`).
 - Detects tag leakage (`<thinking>`) or incomplete structure.

- Classifies responses into `PASS_exact`, `FAIL_thinking_leak`, `PARTIAL_only_think`, and `FAIL_no_schema`.
 - Runs batch validation tests across science, logic, and math queries.
 - Includes interactive terminal REPL with dual-channel response parsing.
-

Notebook 10: Colab Lily-1.5b-v0.3 GGUF Quantization Pipeline

- **Filename:** `workshop_lab/10_colab_lily_1_5b_v03_distill_gguf.ipynb`
 - **Environment:** Google Colab (CPU / T4)
 - **Stage:** Edge Quantization & Binary Packaging
 - **Input:** Model `abhinav0231/Lily-1.5b-v0.3`.
 - **Output Artifact:** Hugging Face Repo `abhinav0231/Lily-1.5b-v0.3-GGUF` containing `F16`, `Q4_K_M`, `Q5_K_M`, `Q8_0` GGUFs.
 - **Working & Code Mechanics:**
 - Clones and compiles `llama.cpp` using CMake (`-DGGML_NATIVE=OFF`).
 - Patches `tokenizer_config.json` with the standardized CoT reasoning ChatML Jinja template.
 - Converts PyTorch model to FP16 GGUF (`convert_hf_to_gguf.py`).
 - Executes K-quantization via `llama-quantize` for `Q4_K_M` (0.98 GB), `Q5_K_M` (1.12 GB), and `Q8_0` (1.62 GB).
 - Conducts live streaming CPU smoke test via `llama-cli` verifying token throughput and `<think>` schema compliance.
 - Uploads all GGUF artifacts to Hugging Face Hub.
-

Notebook 11: Colab Lily-Vision Pretraining Data Preprocessing

- **Filename:** `workshop_lab/11_colab_lily-vision-pretraining-data-preprocessing.ipynb`
 - **Environment:** Google Colab / Modal CPU
 - **Stage:** Phase 1 Projector Pretraining Dataset Ingestion & Tokenization
 - **Input Datasets:**
 - LAION-5K (`laion/relabelled_laion5k`, 5,000 samples)
 - AI2D Science Diagrams (`lmms-lab/ai2d`, 15,000 samples)
 - ChartQA (`lmms-lab/ChartQA`, 28,000 samples)
 - DocVQA (`lmms-lab/DocVQA`, 40,000 samples)
 - InfographicVQA (`lmms-lab/InfographicVQA`, 25,000 samples)
 - **Output Artifact:** Hugging Face Dataset `abhinav0231/lily-pretrain-alignment-dataset` (121,237 samples).
 - **Working & Code Mechanics:**
 - Ingests diverse visual document, chart, diagram, and natural scene datasets.
 - Standardizes image dimensions via `resize_image(image, max_side=448)` and RGB conversion.
 - Inserts special `<image>` token and applies ChatML formatting via `Lily-1.5b-v0.3` tokenizer.
 - Pre-tokenizes text sequences into `input_ids`, `attention_mask`, and `labels` (with prompt text masked to `-100`).
 - Packages raw image bytes and tokenized features into a single high-speed Hugging Face Dataset.
-

Notebook 12: Modal Lily-Vision Projector Pretraining (Phase 1 Alignment v5)

- **Filename:** `workshop_lab/12_modal_lily-vision-projector-pretraining.ipynb`
- **Environment:** Modal Cloud (NVIDIA A100 40GB)

- **Stage:** Vision-Language Projector Alignment
 - **Input:** Vision Tower `google/siglip2-so400m-patch14-384` + LLM `abhinav0231/Lily-1.5b-v0.3` + Alignment Dataset `abhinav0231/lily-pretrain-alignment-dataset`.
 - **Output Artifact:** Trained Projector Weights `abhinav0231/Lily-1.5b-v0.4-Vision-Projector-Pretrained` (`projector_weights.pt`).
 - **Working & Code Mechanics:**
 - Freezes both SigLIP-2 vision encoder and Lily 1.5B LLM.
 - Extracts SigLIP-2 features strictly from the **second-to-last layer** (`hidden_states[-2]`), resolving architecture layer mismatch between pretraining and downstream SFT.
 - Initializes 2-layer MLP Projector (`1152 -> 2048 -> 1536`) with `AdaptiveAvgPool2d((18, 18))` downsampling (compressing 729 vision patches to 324 visual tokens).
 - Trains projector using AdamW ($\eta = 2 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.05) for 1 Epoch (~2,525 steps, $B_{\text{eff}} = 48$).
 - Wraps LR scheduler step in `accelerator.sync_gradients` to ensure proper decay synchronization across accumulated micro-batches.
 - Periodically checkpoints projector weights to Hugging Face Hub with automatic resumption.
-

Notebook 13: Colab Lily-Vision SFT Data Preprocessing & Cleaning

- **Filename:** `workshop_lab/13_colab_lily_vision_sft_data_preprocessing.ipynb`
 - **Environment:** Google Colab / CPU
 - **Stage:** Phase 2 Multimodal SFT Dataset Preparation
 - **Input Datasets:**
 - `MMR1/MMR1-SFT` (Partitions 0 to 5, 48,000 multimodal reasoning samples).
 - `abhinav0231/Sarvam-105b-Distill-100k` (16,000 multi-domain text reasoning samples).
 - **Output Artifact:** Hugging Face Dataset `abhinav0231/Lily-SFT-Clean-Dataset` (64,000 mixed samples: 75% multimodal, 25% text).
 - **Working & Code Mechanics:**
 - Downloads and extracts tar partitions of MMR1-SFT, pairing image files with CoT prompt/response pairs.
 - Cleans Sarvam-105b text responses using `clean_sarvam_answer` (stripping noise, removing metadata artifacts).
 - Filters out incomplete-context prompts (e.g. BCCI agreement legal queries that lack context documents).
 - Assembles unified dataset with multimodal image bytes and text reasoning samples.
-

Notebook 14: Modal Lily-Vision Phase 2 Multimodal SFT Training

- **Filename:** `workshop_lab/14_modal_lily_vision_sft_training.ipynb`
- **Environment:** Modal Cloud (NVIDIA A100 40GB)
- **Stage:** Full Multimodal Instruction Tuning & LoRA Fine-Tuning
- **Input:**
 - LLM Backbone: `abhinav0231/Lily-1.5b-v0.3`
 - Pretrained Projector: `abhinav0231/Lily-1.5b-v0.4-Vision-Projector-Pretrained`
 - Vision Encoder: `google/siglip2-so400m-patch14-384`
 - Mixed SFT Dataset: `abhinav0231/Lily-SFT-Clean-Dataset` (64k samples).
- **Output Artifact:** Merged Full Multimodal Model `abhinav0231/Lily-1.5b-v0.5-Vision-SFT`.
- **Working & Code Mechanics:**

- Applies Unsloth LoRA ($r = 32, \alpha = 64$) across all 7 linear projection layers of the LLM backbone.
 - Loads Phase 1 pretrained MLP projector weights (`projector_weights.pt`), keeping projector trainable ($\eta = 2 \times 10^{-5}$).
 - Freezes SigLIP-2 vision encoder (set to `.eval()` , extracting `hidden_states[-2]`).
 - Utilizes `LengthGroupedBatchSampler` to group variable-length multimodal sequences, accelerating training from 3 hours to ~20 minutes.
 - Employs mixed precision (`bf16`), WandB metric logging, and HF Hub checkpoint callbacks every 150 steps.
 - Merges LoRA adapter weights directly into the LLM backbone at full 16-bit precision and uploads the complete model package.
-

Notebook 15: Colab Lily-Vision Interactive Multimodal Evaluation

- **Filename:** `workshop_lab/15_colab_lily_vision_eval_interactive.ipynb`
 - **Environment:** Google Colab (T4 / GPU)
 - **Stage:** Interactive Multimodal & Text Evaluation
 - **Input:** Multimodal Model `abhinav0231/Lily-1.5b-v0.5-Vision-SFT` .
 - **Working & Code Mechanics:**
 - Implements the lightweight `LilyVLMeval` PyTorch module:
 1. Passes image through SigLIP-2 vision encoder to extract 729 patch features.
 2. Projects pooled features through MLP projector to generate 324 visual embeddings.
 3. Tokenizes prompt and embeds text via LLM embedding lookup.
 4. Splices 324 visual embeddings into the exact position of the `<image>` token.
 5. Feeds spliced multimodal tensor to LLM backbone for autoregressive decoding.
 - Runs batch evaluations across multimodal reasoning queries (charts, geometry, science) and text-only queries.
 - Features an interactive Gradio/Colab UI where users can upload custom images, type prompts, and stream `<think>` reasoning traces and final answers.
-

Notebook 16: Colab Lily-Vision VLM GGUF & `mmproj` Edge Export

- **Filename:** `workshop_lab/16_colab_lily_vlm_gguf.ipynb`
- **Environment:** Google Colab (CPU / T4)
- **Stage:** Edge Quantization & Dual-Binary Multimodal Packaging
- **Input:** Model `abhinav0231/Lily-1.5b-v0.5-Vision-SFT` .
- **Output Artifact:** Hugging Face Repo `abhinav0231/Lily-1.5b-VLM-GGUF` containing:
- LLM Quantizations: `Lily-1.5b-VLM-F16.gguf` , `Lily-1.5b-VLM-Q4_K_M.gguf` , `Lily-1.5b-VLM-Q5_K_M.gguf` , `Lily-1.5b-VLM-Q8_0.gguf` .
- Multimodal Projector: `mmproj-Lily-1.5b-VLM-f16.gguf` .
- **Working & Code Mechanics:**
- Compiles `llama.cpp` with multimodal tools enabled (`-DGGML_NATIVE=OFF`).
- Injects standardized ChatML reasoning Jinja template into `tokenizer_config.json` .
- Converts language model backbone to FP16 GGUF (`convert_hf_to_gguf.py`).
- Exports SigLIP-2 vision tower and 2-layer MLP projector into `mmproj` GGUF binary format via `convert_image_encoder_to_gguf.py` .
- Performs K-quantizations (`Q4_K_M` , `Q5_K_M` , `Q8_0`) using `llama-quantize` .

- Conducts live streaming smoke tests for text-only reasoning and multimodal image reasoning (`llama-cli --mmproj`).
- Uploads complete GGUF package to Hugging Face Hub for edge deployment.

10. Cloud Checkpointing & Fault-Tolerant Auto-Resumption

Across all cloud training notebooks (`02` , `04` , `07` , `12` , `14`), running on serverless infrastructure (Modal) presents a risk of preemption or timeout. We built custom Hugging Face Hub callbacks that automatically snapshot checkpoints to private repos and auto-resume on restart.

```
from huggingface_hub import HfApi, snapshot_download
import os

# Auto-Resume Logic across Training Notebooks (02, 04, 07, 12, 14)
if RESUME_FROM_CHECKPOINT and CHECKPOINT_REPO:
    api = HfApi()
    files = list(api.list_repo_files(CHECKPOINT_REPO, token=HF_TOKEN))
    ckpt_nums = [int(f.split("/")[0].split("-")[-1]) for f in files if "checkpoint-" in f]
    if ckpt_nums:
        latest = max(ckpt_nums)
        local_dir = os.path.join(OUTPUT_DIR, f"checkpoint-{latest}")
        print(f"Downloading latest checkpoint-{latest} from HF Hub: {CHECKPOINT_REPO} ...")
        snapshot_download(
            repo_id      = CHECKPOINT_REPO,
            allow_patterns = [f"checkpoint-{latest}/*"],
            local_dir     = OUTPUT_DIR,
            token         = HF_TOKEN,
        )
        _resume_ckpt = local_dir
```

Conclusion & Summary Checklist

By combining **PEFT (LoRA/QLoRA)**, **Hyperparameter Scaling Equations**, **GRPO Reinforcement Learning**, **Offline SFT Distillation**, **GGUF K-Quantization**, **2-Phase Multimodal VLM Alignment**, and **Dual-Binary Edge Packaging** (`mmproj`), our 16-notebook codebase establishes a complete, production-grade LLM and VLM development framework.